

9주차 - 구조체

컴파일러

C언어는 컴파일러 언어이기 때문에 한 줄씩 번역하지 않고, 모든 소스를 번역 후 실행파일로 생성

소스코드(.c) --전처리기--> 전처리소스(.i) --컴파일러--> 어셈블리코드(.s) --어셈블러--> 오브젝트파일(.o)
--링커--> 실행파일(.exe)

ctrl+alt+N : 한번에 실행 가능!

문자

프로그래밍 언어에서 자료를 식별하는 종류

컴퓨터는 항상 2진수 정보만을 인식 --> 이를 어떻게 표현/식별할 것인가를 정의

기본형(basic types), 유도형(derived types), 사용자 정의형(user defined types)
정수형(short, int, long), 실수형(float, double), 문자형(char), 무치형(void 등...)

문자형 변수(char)

문자 또는 1바이트 크기의 정수를 저장하기 위한 변수

C언어는 ASCII code를 기준으로 문자를 처리

초기 ASCII C0de 7-bits 0~127까지 128개의 문자를 정의

이후 Extended ASCII Code 8-bits 0~255까지 256개의 문자를 정의

A-65 a-97

문자열

문자로 이루어진 배열(각 배열의 원소마다 하나의 문자)

문자열의 초기화

char str1[] = "Hello World" -> 일반적인 방법

char str2[] = {'H', 'e', 'l', 'l', 'o', ... 'd'};

문자열의 크기 : [글자의 수]+1 마지막에 NULL 문자를 저장해야 하기 때문!

문자열 관련 함수

#include <string.h> 문자열 관련 함수 - strlen() 등...

#include <stdio.h> 표준입출력 관련 함수를 포함시킴!

“배열의 이름은 배열의 첫 번째 원소의 주소값이다.”

strcmp()

strncmp() 등등... 한번 복습하기!

구조체

정수, 문자, 실수나 포인터 그리고 이들의 배열 등을 묶어 하나의 자료형으로 이용하는 것
관련 있는 정보들을 묶어서 처리/관리

정수, 문자, 실수나 포인터 그리고 이들의 배열 등을 묶어 하나의 자료형으로 이용하는 것

구조체의 정의 : 표현하고자 하는 정보를 묶어, 하나의 자료형으로 정의

```
struct student{
    char name[20];
    int year;
    char major[20];
    char address[50];
    double grade_programming;
    double grade_oss_python;
}
```

구조체 변수의 선언과 접근 : 변수 선언과 동일한 방법 (자료형+변수이름)

※ 자료형의 이름은 struct까지 포함!

구조체 멤버의 접근은 . 연산자로

```
struct student puang;
strcpy(puang.name, "Puang");
puang.year = 2021;
puang.grade_programming = 4.5;
printf("Year of Puang: %s\n", puang.year);
```

구조체 변수의 초기화

1. 중괄호로 묶어 데이터를 나열(배열과 유사)

```
struct student puang = {"Puang", 2021, "Software", "Incheon, Korea", 4.5, 4.0};
```

2. 중괄호 내부에 멤버를 명시하면 더욱 깔끔

```
struct student chung_ang = {
    .year = 1916,
    .name = "Chung-Ang",
    .major = "Software",
    .address = "Seoul, Korea",
    .grade_oss_python = 3.5,
    .grade_programming = 4.0
};
```

배열

- 관련있는 데이터를 묶어서 관리
- 중괄호를 이용하여 초기화
- 모든 원소가 **동일한 타입**으로 한정
- 선언할 때 원소의 **개수를 지정** 가능
- **연속된 메모리**에 저장

```
double grades[4] = {4.5, 4.0, 3.5, 4.0};
```

.....▶ week9_ex3.c

구조체

- 관련있는 데이터를 묶어서 관리
- 중괄호를 이용하여 초기화
- 각 멤버마다 **다른 타입**으로 생성 가능
- 타입이 지정되면 멤버의 **구성은 고정**
- **연속되지 않은 메모리**에 저장될 수 있음
(기본적으로는 연속된 메모리)

```
struct student {
    char name[6]; int studentid; double grade;
};
struct student puang = {"Puang", 20210000, 4.5};
```

배열 vs 구조체

Word : 연산의 기본 단위가 되는 크기

packing : 구조체에서는 **가장 크기가 큰 멤버**의 배수로 word를 지정하는 것

padding : 남은 공간은 비워 두는 것

하나의 변수에 접근하기 위해 메모리에 2번 접근 해야하는 경우가 발생 → **속도 저하**

멤버 변수의 크기만큼만 메모리 사용 → **메모리 효율**

장단이 있다!

구조체 포인터

구조체 포인터의 생성과 사용

일반 변수와 동일한 방법으로 포인터 변수 **생성/사용** 가능

구조체 포인터에서 멤버에 접근하기 위해서는 구조체 **접근(*)**에 괄호를 사용

(*p).name

우선순위 : 기본적으로 괄호를 안쓰면 **멤버접근** 후 **포인터접근**

p->name : 이 방식은 **포인터 접근** 후 **멤버접근**을 할 수 있음.

typedef 자료형 재정의

자료형의 이름을 간결하게 바꾸고 싶을 때

구조체 변수를 만들기 위해서 **구조체 자료형**을 쓰는 것은 귀찮음.

가끔 **너무 긴 이름의 자료형**이 있을 수 있음.

호환성을 고려할 때

16비트 운영체제와 64비트 운영체제의 int 자료형의 크기가 다름. e.g. 2bytes vs 4bytes

=> 자료형의 이름을 새롭게 만들어 **간결하고 확실한 크기의 타입**을 정의.

```
typedef [original_type] [new_type];
```

e.g.

```
typedef struct student student
```

```
struct student puang; → student puang;
```

```
typedef unsigned long long int ullint;
```

```
unsigned long long int score; → ullint score;
```

구조체의 정의와 동시에 typedef로 재정의

```
typedef struct _student_t {
```

```
char *name;
```

```
int studentid;
```

```
double grade;
```

```
} student_t;
```

구조체 + typedef

- 구조체의 **정의와 동시에** typedef로 재정의

```
typedef struct [original_type] {  
    type1 member1;  
    type2 member2;  
    type3 member3;  
} [new_type];
```

```
typedef struct student {  
    char *name;  
    int studentid;  
    double grade;  
} student;
```

구조체를 생성할 때는
[original_type] 생략 가능(비추천)

TIP. 새로운 타입을 정의할 때 특정 [type]을 만들 때

[new_type]의 이름은 [type]_t

[original_type]의 이름은 **_[type]_t**로 사용(그냥 관례처럼...)

```
typedef struct _student_t {  
    char *name;  
    int studentid;  
    double grade;  
} student_t;
```

.....→ week9_ex10.c

이름없는 구조체 (시험X)

이름을 쓰지 않고 변수 생성함과 동시에 초기화

공용체 Union (시험X)

멤버 변수들이 같은 메모리를 공유하는 자료형

열거형 Enum

정수에 이름을 붙여서 사용하고자 할 때.

```
enum ops {  
    ADD, SUB, MUL, DIV, POW  
};
```

10주차 - 파일 입출력

파일

: 프로그램에서 수행 혹은 메모리에 저장한 내용

프로그램의 종료와 함께 사라짐

디스크(Disk)에 파일로 저장하면 직접 삭제하지 않는 한 유지

파일을 사용하는 경우?

프로그램에서 수행한 **결과 값을 저장**하고 싶을 때

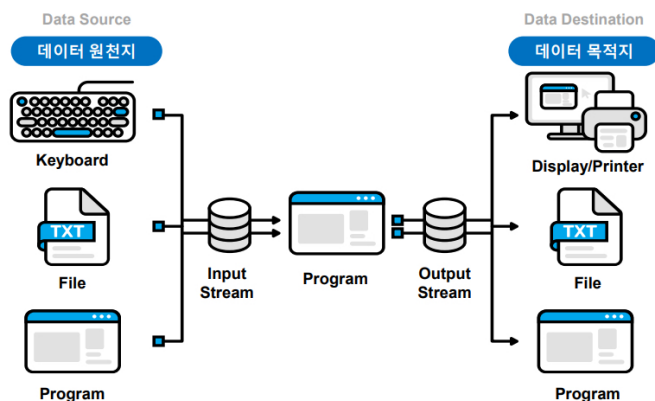
이미 존재하는 데이터를 **불러들여 처리**하고 싶을 때

스트림

: 자료의 흐름/이동 경로

스트림 (Stream)

• 자료의 흐름/이동 경로



입력 스트림(Input stream)

- 표준입력(Keyboard)
- 파일입력(File)
- 스크린입력(Screen)
- 네트워크입력(Network)

출력 스트림(Output stream)

- 표준출력(Console)
- 파일출력(File)
- 프린터출력(Printer/paper)
- 네트워크출력(Network)

파일 입출력 함수

파일 스트림 열기/닫기

fopen(), fclose()

텍스트 입출력

fscanf(), fprintf(), fgets(), fputs()

이진(binary) 입출력

fread(), fwrite()

파일 스트림 열기/닫기

`FILE *fopen(const char *filename, const char *mode)`

반환 값 : 파일 내용의 시작 주소(파일 포인터), 파일이 없거나 읽기 실패할 경우 NULL 반환

매개변수 : 파일의 경로 및 이름(filename), 모드(읽기-r, 쓰기-w, 추가-a, 이진-b)

`int fclose(FILE *stream);`

반환 값 : 성공 시 0, 실패 시 EOF

매개변수 : 파일 포인터

FILE 개요

`FILE *var` : 파일의 메모리를 가리키는 포인터, 타입은 대문자로 표기

mode : 읽기-r, 쓰기-w, 추가-a, 이진-b

읽고+쓰기모드 : 이전내용 삭제 w+ 마지막 내용에 붙여서 a+ 파일존재여부 체크 r+

EOF(End of File) : 파일의 끝 혹은 파일처리에서 에러를 나타내는 값.

파일 입출력 함수

파일 읽기

// 띄어쓰기 만나면 끝남!

`int fscanf(FILE *stream, const char *format, ...);`

// 띄어쓰기 무시 줄바꿈문자 만날 때까지 n개 읽기

`char* fgets(char * str, int count, FILE* stream);`

파일 쓰기

// 출력해 stream에 저장하기 printf랑 같으나, 맨 앞 인자가 파일 포인터

`int fprintf(FILE *stream, const char *format, ...);`

// 띄어쓰기 무시 줄바꿈문자 만날 때까지 n개 읽기

`char* fputs(char * str, int count, FILE* stream);`

fprintf와 fscanf는 파일포인터 인자가 맨 앞에,

fgets와 fputs는 파일포인터 인자가 맨 뒤에 온다!

파일의 종류

텍스트 파일(Text file)

문자로 저장되어 있는 파일(.txt, .csv 등)

텍스트 편집기를 사용하여 내용 열람 및 수정 가능

이진 파일(Binary file)

목적에 맞게 자료가 이진(binary) 형태로 저장(사진, 음악, 동영상, PDF, 실행파일 등)

입출력 속도가 빠르나, 데이터의 구조를 알고 있어야 해독 및 수정이 가능

해당 확장자 형식을 읽을 수 있는 파일/편집기로 열람 및 수정 가능

`fp = fopen(fname, "rb")`

rb옵션 : 이진 파일을 읽기 모드로

바이너리 모드 입출력 함수

파일 읽기(바이너리 모드)

- `size_t fread(void *buffer, size_t size, size_t count, FILE *stream);`

ptr: 데이터를 **저장**할 메모리의 시작 주소입니다.

size: 읽어올 데이터의 **요소 한 개의 크기**(바이트 단위)입니다.

nmemb: 읽어올 **요소(element)의 개수**입니다.

stream: 파일 스트림(FILE *)입니다.

반환값

읽은 요소의 개수를 반환합니다(요소 단위). 읽은 개수가 0이면 파일의 끝에 도달!

파일 쓰기(바이너리 모드)

- `size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);`

ptr: 저장할 데이터가 있는 메모리의 시작 주소입니다.

size: 저장할 데이터의 한 개 요소 크기(바이트 단위)입니다.

nmemb: 저장할 요소의 개수입니다.

stream: 파일 스트림(FILE *)입니다.

반환값

파일에 쓰여진 요소의 개수를 반환합니다(요소 단위). 쓰기 실패 시, 반환값은 0이 될 수 있습니다.

buffer 사이즈를 적절하게 결정해야 한다!

적게하면 할수록 오류(중간에 잘릴 가능성)는 줄어듦. 크게하면 할수록 속도는 빨라짐.

11주차 - 동적할당

가변길이 배열 VLA

- : 실행 중(Runtime) 배열을 선언할 때 크기에 변수의 값을 넣어 **동적으로 메모리를 할당**
- 기존의 배열의 선언/사용과 동일한 방법으로 사용가능

단, 추천하지 않는 기능.

다른 표준/컴파일러와의 **호환성 문제** 有
스택메모리(Stack)의 한계

컴파일러의 종류와 역사

하나의 언어에 대한 컴파일러의 종류도 여러 개가 존재(gcc, clang, msvc 등)
한 종류의 컴파일러도 버전에 따라 지원되는 표준이 상이(c89, c90, c99, c11, c17 등)
수업시간에 허용되었던 몇몇 간편한 기능들은 C99 이후 등장한 기능
한 종류의 컴파일러로부터 파생된 새로운 컴파일러들도 존재...

메모리 영역 구조

스택 메모리

함수가 호출될 때마다 스택에 적재
매개변수, 함수 내 지역변수 등...

스택 메모리는 매우 적다!

->제한을 풀 수는 있지만 추천X

변수 유효범위

선언된 변수가 사용될 수 있는 범위

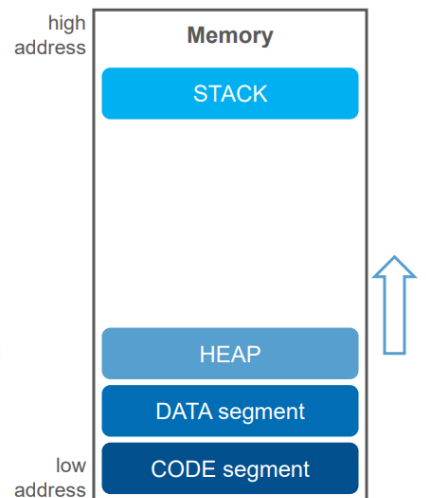
지역 범위 : 특정 지역에서만 생성되고 참조가 가능한 변수

번역 범위 : 여러 블록이나 함수에서 접근할 수 있는 변수

-> 함수의 밖에 구현되면 전역변수, 안에 있으면 지역변수

메모리 영역 구조

- **코드 영역 (Code segment, Text segment)**
 - 작성한 코드가 기계어로 번역되어 저장
- **데이터 영역 (Data segment)**
 - 전역변수, 정적변수 등이 저장
- **스택 영역 (Stack)**
 - 함수의 호출, 매개변수, 지역변수 등이 저장
- **힙 영역 (Heap)**
 - 동적할당과 관련된 메모리가 저장



동적 메모리 할당

#include <stdlib.h> 동적메모리 관리하기 위해서는 stdlib.h 헤더파일을 써야 함.

void *malloc(size_t size);

//size 바이트의 메모리를 할당하여, 힙 영역에 확보.

//void 포인터의 반환형이라 모든 포인터 타입과 호환되는 자료형.

//반환 : 실패 시 NULL / 성공 시 할당된 메모리의 시작 주소

공간을 만들어서 누군가(포인터 변수)에게 공간 만들어 놨다고 알려주는 것!

void free(void *memblock) //memblock의 메모리를 해제하여 반환

HEAP 메모리의 동적할당은 시점이 불명확함.

메모리를 다 사용한 후 반환하지 않으면 프로그램의 메모리 사용량이 급격히 증가함.

2차원 메모리 동적할당 해제할 때는 순서에 신경써야 함!

해제는 할당의 역순.

메모리 관련 함수

<string.h> 안에 있음.

```
void *memset(void *dest, int c, size_t count);
```

dest 메모리에 c값을 count만큼 채움 (이때, c는 unsigned char로 형변환)

```
int memcmp(const void *buffer1, const void *buffer2, size_t count);
```

buffer1 메모리에 있는 값과 buffer2 메모리에 있는 값을 count만큼 비교

```
void *memcpy(void *dest, const void *src, size_t count);
```

src 메모리의 값을 count만큼 dest 메모리에 복사

```
void *memmove(void *dest, const void *src, size_t count);
```

src 메모리의 값을 count만큼 dest 메모리에 복사

12주차 - 전처리와 라이브러리

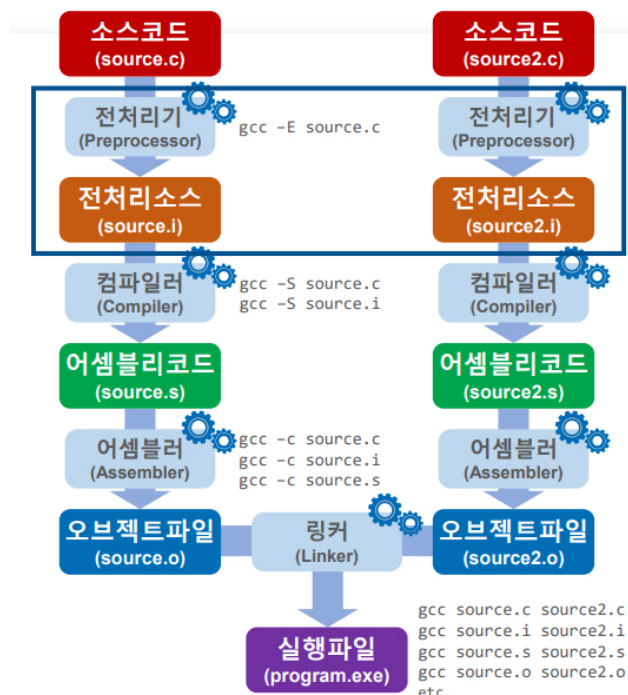
전처리(Preprocessor)

컴파일 이전에 소스코드를 대체하여 코드를 재구성하는 기능
#을 이용하여 구현

```
#define / #undef  
#ifdef / #ifndef  
#if / #elif / #else  
#endif  
#error  
#include
```

소스코드로 전처리 소스 만들기

```
gcc -E source.c -o pp_source.i
```



#define

특정 내용을 정의(상수 or 작업 등)
매크로라고도 불림
소스코드를 대체!

사용법

#define [매크로] 구문

매크로 : 대체할 구문
구문 : 상수, 함수 등...

#undef

정의한 내용을 삭제

괄호를 적절하게 사용하고, if 문 등은 do while 문으로 대체한다.

매크로 상수 : 특정한 값에 이름을 부여할 때 사용. 특별한 규칙 없이 비교적 간단

매크로 함수 : 결과값을 받아올 함수. 내용을 소괄호로 묶음. 여러 내용을 수행하는 용도(SWAP 등).

내용을 do { } while(0)으로 묶는다.

#ifdef #ifndef #else #endif

전처리 과정에서 특정 키워드가 **define** 되었는지 **안** 되었는지 **비교**. 값에 상관X
해당 조건에 부합하는 내용만 처리. 나머지는 없는 소스로 처리

```
#ifdef [KEYWORD]
```

```
#else
```

```
#endif //endif를 반드시 써 줘야 한다.
```

```
#ifdef
```

값에 관계없이 정의가 되었는지 확인.

다양한 경우에 쓰이지만, 대표적으로 디버깅.

컴파일 단계에서 특정 매크로를 정의하려면 -D 옵션을 사용한다.

```
* gcc -o [run.exe] [source.c] -D[MACRO_NAME]
```

```
ex) gcc -o week12_ex17.exe week12_ex17.c -DDEBUGON
```

디버깅할 때 사용 가능 12-17, 12-18 참고

#if #elif #else #endif #error

전처리 과정에서 조건을 비교

해당 조건에 부합하는 내용만 처리

나머지는 없는 소스로 처리

```
#if [condition1]
```

```
#elif [condition2]
```

```
#elif [condition3]
```

```
#else
```

```
#endif
```

컴파일 단계에서 -D 옵션으로 매크로 값을 지정할 수 있다.

```
* gcc -o [run.exe] [source.c] -D[MACRO_NAME]=[VALUE]
```

```
ex) gcc -o week12_ex17.exe week12_ex17.c -DSYSTEM=CONTIKI
```

라이브러리

재사용이 가능한 기능들을 제공하는 바이너리파일

컴파일된 함수들의 집합 파일 정도로 이해

주요 확장자: .lib, .dll, .a, .so 등

정적라이브러리

컴파일 단계에서 라이브러리의 내용을 **내 프로그램에 복사**

헤더파일 등

동적라이브러리

컴파일은 내 코드만!

(내용은 다른 메모리에 존재, 필요할 때에만 접근) -> 요즘은 동적 라이브러리를 더 많이 씀

헤더파일

공통으로 사용되는 변수, 함수, 매크로 등이 선언된 파일

함수의 구현내용도 정의할 수 있지만 보통은 선언만 해두고 다른 소스 혹은 라이브러리에 구현

큰 프로젝트에서 작성할 때 유용 +협업

코드를 확인하고 수정할 수 있도록 라이브러리가 아닌 코드로 프로젝트를 만들고자 한다.

프로젝트는 WiFi 통신으로 데이터를 전송하는 프로그램이다.

A개발자는 데이터를 WiFi를 통해 전송하는 기능을 개발한다. (wifi_tx 함수 개발)

B개발자는 WiFi를 통해 데이터를 수신하는 기능을 개발한다. (wifi_rx 함수 개발)

C개발자는 수신한 데이터를 처리하고 새로운 데이터를 생성한 후 다시 전송하는 작업을 개발한다. (함수 사용, main)

하나의 소스파일에 모든 것이 개발하면 번거롭다...

※주의 : 시스템에 있는 헤더파일은 #include <stdio.h>로 불러오고 직접 만든 건 #include "wifi.h"로 씬
꺾쇠와 쌍따옴표 구분!

gcc -o main.i -E main.c //main.c 파일을 main.i 파일로 전처리할래

```
1> gcc -o hello.o -c main.c
1> gcc -o tx.o -c wifi_tx.c
1> gcc -o rx.o -c wifi_rx.c
1> gcc hello.o tx.o rx.o
1> gcc hello.o tx.o rx.o -o hello.exe
1> .\hello.exe
```

헤더파일 중복문제

- main.c 파일에서 헤더파일B와 헤더파일A를 호출하고 있다.
- 헤더파일B는 헤더파일A를 호출하고 있다.

#ifndef와 #define으로 딱 한 번만 수행되게 변경!

13주차 - 자료구조

자료구조(Data Structure)

프로그램 = 자료구조 + 알고리즘

자료구조

데이터를 효율적으로 저장하고, 꺼내기 위한 수단

자료구조의 선택 -> 프로그램의 속도&메모리 효율을 좌우

사전에서 책갈피가 없으면 찾기가 힘들...

리스트

데이터가 순차적으로 (선형적으로) 저장되어 있는 자료구조

삽입, 삭제, 탐색에 대한 기능을 지원

리스트의 종류

배열리스트

연결리스트

원형연결리스트

이중연결리스트...

리스트 자료구조 구현을 위한 함수

create/destroy : 리스트를 생성 소멸
insert/delete : 리스트에 원소를 추가/삭제하는 함수
search : 리스트에서 값을 찾는 함수

insert/delete

insert_first/delete_first : 첫 번째 요소를 추가/삭제
insert_last/delete_last : 마지막 요소를 추가/삭제
insert_index/delete_index : index번 요소에 추가/삭제

search

search_by_value : 리스트에 해당 값이 존재하는지 확인하고, 그 값을 반환
search_by_index : 특정 인덱스의 요소를 반환

Array List

배열을 사용하여 구현하는 리스트

장점 : 구현이 매우 간단, 탐색이 빠름

단점 : 삽입/삭제 시 원소이동에 시간 소요

Array List의 구현

데이터 저장을 위한 Array List 자료구조

리스트의 길이

원소들 -> 배열 ; 배열을 가리키는 포인터

```
typedef int element_t;
```

```
typedef struct _array_list_t {  
    unsigned int len;           //길이  
    element_t *elements;       //포인터. 근데 왜 element_t이지? 그냥 정수형. 가끔 다른 타입이 될수도  
} array_list_t;
```

Array List 생성

14주차 - 정렬 알고리즘

알고리즘

프로그램 = 자료구조 + 알고리즘

알고리즘 Algorithm

문제의 의미를 파악하고, 목적을 달성하기 위한 단계별 절차

Algorithm : 숫자를 이용하여 연산하는 기술

By **problem or topic**

Sorting , Searching, Selection, Graph, Tree, String -> 수업에서 다룰 것

By **approach or method**

Divide and conquer, Dynamic Programming, Greedy, Randomized, Approximation, Backtracking

정렬

연속된 데이터를 기준에 따라 **순차적으로 정리하는 기술**

정렬을 하는 이유? - 처리를 위한 많은 알고리즘이 이미 정렬된 데이터에서 시작하는 경우가 다수

e.g. 책에서 원하는 페이지를 열 때, 사전에서 원하는 단어를 찾을 때

선형탐색 vs 이진검색

정렬 알고리즘의 종류

Selection Sort 선택 정렬

Insertion Sort 삽입 정렬

Merge Sort 병합 정렬

Selection Sort 선택 정렬

기준에 가장 부합하는 데이터를 선택하여 순차적으로 나열

최소값부터 하나씩 큰 값을 찾고, 그 위치와 아닌 위치를 SWAP

```
void selection_sort(int *data, int n) {
    int i;          // 몇 번째 최소값을 찾을 것인지
    int j;          // 최소값을 탐색을 위해 움직일 변수
    int min_idx;    // 1번의 탐색에서 현재까지의 최소값의 인덱스를 저장할 변수
    int temp;       // swap을 위한 변수

    for (i = 0; i < n - 1; i++) {
        min_idx = i;          // 탐색을 시작할 때, 현재까지 확인된 최소값의 인덱스는 i
        for (j = i + 1; j < n; j++) {      // 다음부터 끝까지 탐색
            if (data[j] < data[min_idx]) { // 최소값을 찾으면
                min_idx = j;              // 최소값의 인덱스를 업데이트
            }
        }
        // 최소값을 찾았으면, i번째와 최소값을 swap
        temp = data[i];
        data[i] = data[min_idx];
        data[min_idx] = temp;
    }
}
```

Insertion Sort 삽입 정렬

이미 정렬이 완료된 리스트에 새로운 데이터가 추가된다면? -> 삽입 정렬이 제일 적절
앞으로 가면서 나보다 크면 바꾸기

그때그때 SWAP

```
void insertion_sort(int *data, int n) {
    int i;    // 정렬할 데이터의 인덱스
    int j;    // 비교할 데이터의 인덱스
    int temp; // swap을 위한 변수

    for (i = 1; i < n; i++) {
        for (j = i; j >= 1 && data[j-1] > data[j]; j--) {
            temp = data[j-1];
            data[j-1] = data[j];
            data[j] = temp;
        }
    }
}
```

옮길 것 빼놓고 하나씩 뒤로밀기

```
void insertion_sort(int *data, int n) {
    int i;    // 정렬할 데이터의 인덱스
    int j;    // 비교할 데이터의 인덱스
    int key;  // 비교할 데이터를 저장할 변수

    for (i = 1; i < n; i++) {
        key = data[i];           // 삽입할 데이터를 key에 저장
        for (j = i; j >= 1 && data[j-1] > key; j--) // 적절한 위치를 찾을 때까지
            data[j] = data[j-1]; // 뒤로 이동
        data[j] = key;           // 삽입할 데이터를 삽입
    }
}
```

memmove를 사용. 한번에 옮기기

```
void insertion_sort(int *data, int n) {
    int i;    // 정렬할 데이터의 인덱스
    int j;    // 비교할 데이터의 인덱스
    int key;  // 비교할 데이터를 저장할 변수

    for (i = 1; i < n; i++) {
        key = data[i];           // 삽입할 데이터를 key에 저장
        for (j = i; j >= 1 && data[j-1] > key; j--) // 적절한 위치를 찾을 때까지 j를 이동하며
            // 아무 작업도 하지 않음
        /** 이 시점에 j는 key가 들어갈 위치를 가리키고 있음 **/
        memmove(data + j + 1, data + j, sizeof(int) * (i - j)); // 한 번에 밀어내기
        data[j] = key;           // 삽입할 데이터를 삽입
    }
}
```

이미 정렬되어 있는 상태

- Selection Sort
 - N-1번, N-2번, N-3번 ... 1번 $\rightarrow N(N-1)/2 \approx N^2$
- Insertion Sort
 - 1번, 1번, 1번 ... 1번 $\rightarrow (N-1) \times 1 \approx N$

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

거꾸로 정렬되어 있는 상태

- Selection Sort
 - N-1번, N-2번, N-3번 ... 1번 $\rightarrow N(N-1)/2 \approx N^2$
- Insertion Sort
 - 1번, 2번, 3번 ... N-1번 $\rightarrow N(N-1)/2 \approx N^2$

10	9	8	7	6	5	4	3	2	1
----	---	---	---	---	---	---	---	---	---

랜덤

- Selection Sort
 - N-1번, N-2번, N-3번 ... 1번 $\rightarrow N(N-1)/2 \approx N^2$
- Insertion Sort
 - 다음 원소가 작을 확률 $\frac{1}{2}$, 클 확률 $\frac{1}{2} \rightarrow N(N-1)/4 \approx N^2$

8	3	5	6	2	1	9	4	10	7
---	---	---	---	---	---	---	---	----	---

Selection Sort



Insertion Sort



Selection Sort



Insertion Sort



Selection Sort



Insertion Sort



Selection Sort는 데이터의 배치와 상관없이 N^2 으로 동일하다

Insertion Sort는 데이터가 잘 정렬되어 있을수록 속도가 빠르다. 최소 N 최대 N^2

Merge Sort 병합 정렬

이미 정렬이 완료된 2개의 리스트를 합쳐서
새로운 정렬된 리스트를 만들고자 한다.
이런 상황에서 이상적!

각 리스트의 길이가 1이 될 때까지 쪼갬
다시 2의 배수씩 합병하면서 Merge sort 수행

추가 공간이 필요함. N개 배열이면 N개짜리 필요함. 제일 큰 단점

```

void merge_sort (int *arr, int N) {
    int i, j, k;    // 탐색을 위한 변수
    int *temp_arr = NULL;

    // 길이가 1일 경우 이미 정렬 완료된 리스트로 가정
    if (N == 1) return;

    // 이등분하여 각각 정렬수행
    merge_sort(arr, N/2);
    merge_sort(arr + N/2, N - N/2);

    temp_arr = (int *)malloc(sizeof(int) * N); // 임시 저장 공간 생성
    // 정렬된 2개의 리스트에 대해 정렬 수행
    i = 0;    j = N/2;    k = 0;
    while (i < N/2 && j < N) {
        if (arr[i] > arr[j])
            temp_arr[k++] = arr[j++];
        else
            temp_arr[k++] = arr[i++];
    }

    // 남아있는 원소들 전부 복사
    while (i < N/2) temp_arr[k++] = arr[i++];
    while (j < N)   temp_arr[k++] = arr[j++];

    // 임시 배열에서 원본 배열로 복사
    memmove(arr, temp_arr, sizeof(int) * N);

    // 메모리 해제
    free(temp_arr);
}

```

Merge Sort (합병정렬) – 정수 오름차순

- N개의 원소를 비교 및 탐색하는 횟수 → N
- 2등분하는 단계의 깊이 → $\log_2 N$

